

AI Fraud Risk Investigation App

A simple dashboard that helps a fraud analyst investigate a suspicious card transaction quickly and clearly.

In one sentence:

The analyst enters a Transaction ID, and the app combines transaction data, customer history, and bank policy to explain the risk and suggest the next action.

Project: RiskManagement

Presentation overview · Business use case and current prototype

1. The Problem

Fraud analysts usually need to look in several places before deciding whether a transaction is suspicious:

Transaction system

Check amount, merchant, location, time, card status, and IP information.

Customer history

Check the customer's normal spending and previous transactions.

Risk policy document

Search a large policy PDF to find the rules that apply.

Manual decision

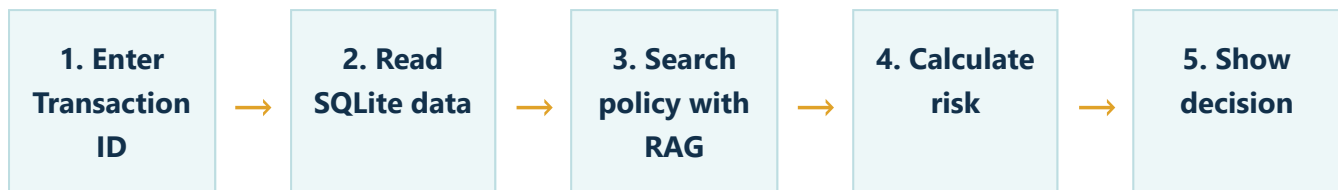
Combine all evidence and decide whether to monitor, review, or freeze the card.

Business goal: replace these separate steps with one investigation workspace that is faster, clearer, and easier to explain.

What the app provides

- A transaction and customer overview
- Automatically detected risk indicators
- A calculated risk score from 0 to 100
- Relevant policy sections and source pages
- A plain-language executive summary
- Buttons for Freeze Card, Notify Customer, and Escalate

2. How the App Works



Simple workflow

1. The analyst chooses a transaction such as `TXN_99812`.
2. The Flask backend finds the transaction in SQLite.
3. It finds the customer profile and transaction history.
4. It checks risk signals such as unusual amount, proxy IP, international location, and rapid location changes.
5. ChromaDB searches the bank policy PDF for matching rules.
6. The dashboard displays the score, findings, policy citations, summary, and next actions.

Example results

Transaction	What it shows	Result
TXN_99812	Large purchase, proxy IP, international activity, and rapid cross-location activity.	High Risk Immediate freeze
TXN_998112	Small purchase, normal pattern, and clean residential IP.	Low Risk Normal monitoring

Important: the numeric risk score is calculated by application rules. The language model summarizes the evidence; it does not choose the score.

3. Technologies Used

Technology	What it does in this project
Python	Main programming language for the backend and risk logic.
Flask	Runs the web server and provides the API endpoints.
SQLite	Stores demo customer profiles and transaction records.
ChromaDB	Stores the policy PDF as searchable vector data.
RAG	Retrieval-Augmented Generation: finds relevant policy text before creating the summary.
PyPDF	Reads and extracts text from the bank risk policy PDF.
HTML, CSS, JavaScript	Builds the analyst dashboard in the browser.
OpenAI / Gemini	Optional language-model providers for generating the investigation summary.
Python fallback	Provides a deterministic summary when no AI provider is configured or available.

Why these technologies?

Fast to build

Python and Flask make it easy to connect data, AI services, and a browser interface.

Grounded answers

RAG connects the summary to the bank policy instead of relying only on model memory.

Easy to demonstrate

SQLite and local ChromaDB allow the prototype to run on a laptop.

Human decision support

The app explains evidence and recommends actions while keeping the analyst in control.

4. Risk Score and AI Design

Dynamic scoring

The score is generated from transaction values and customer context rather than assigning a score from the Transaction ID.

Risk signal	Score added
Transaction is 2x, 5x, or 10x above the customer average	+10, +18, or +25
Proxy IP detected	+15 or +30 when amount is above \$500
Transaction is outside the customer's home country	+10
Card-not-present transaction	+5
Physical transaction follows online activity in another location within two hours	+70

0–39

LOW RISK

Normal monitoring

40–79

MEDIUM RISK

Secondary review

80–100

HIGH RISK

Immediate freeze

Three AI building blocks

1. Tools / data lookup

SQLite provides factual transaction and customer information.

2. RAG policy search

ChromaDB retrieves relevant sections from the policy PDF.

3. LLM summary

Grounding

OpenAI or Gemini turns the evidence into a concise analyst-friendly explanation.

The response includes findings, policy sections, source pages, and an execution trace.

5. Demo and Business Value

Suggested presentation demo

1. Start the app with `python app.py`.
2. Open `http://127.0.0.1:5000`.
3. Investigate `TXN_99812` and show the High Risk findings and policy evidence.
4. Investigate `TXN_998112` and compare the Low Risk result.
5. Show the execution trace to explain the SQLite and ChromaDB steps.
6. Click an action button and show the updated transaction status.

Business value

Faster

One dashboard replaces multiple manual lookups.

More consistent

The same scoring rules and policy search path are used for each transaction.

Explainable

Analysts can see the data and policy evidence behind a result.

Actionable

The result includes clear next steps instead of only a prediction.

Current prototype and next steps

The current version is a local hackathon prototype with seeded data, a lightweight offline embedding function, and optional AI providers. A production version would add secure bank-system connectors, access control, audit logging, policy versioning, calibrated scoring, and stronger model monitoring.

Closing message: this project shows how AI can help a fraud analyst move from “searching for evidence” to “reviewing an explainable decision.”